

Hardware Root of Trust in RISC-V Systems: Analysis and Evaluation of Open Source Approaches

Pau Romaguera Palomé

December 11, 2025

Resum– L'evolució ràpida del hardware d'altres prestacions ha transformat la indústria dels semiconductors, possibilitant avenços en intel·ligència artificial (IA) i computació d'altres prestacions (HPC). A mesura que aquests sistemes esdevenen més sofisticats, garantir la seguretat del xip és essencial per protegir les dades que processen. Aquesta tesi avalua funcionalitats de seguretat de codi obert en sistemes basats en RISC-V mitjançant proves adversàries, centrant-nos en mecanismes de protecció de memòria com la protecció física de memòria (PMP) i la PMP millorada (ePMP), en relació al Secure Boot. Prenent com a cas d'estudi el System on Chip (SoC) anomenat Earlgrey d'OpenTitan [1] [2] [3], afeblim o eliminem proteccions seleccionades per quantificar la contribució de cada mecanisme i derivar recomanacions pràctiques per enfortir dissenys de Hardware Root of Trust (HRoT).

Paraules clau– Computació d'altres prestacions (HPC), plataformes RISC-V, intel·ligència artificial (IA), sistemes arrelats en hardware (HRoT), Protecció Física de Memòria (PMP)

Abstract– The rapid evolution of high-performance hardware has transformed the semiconductor industry, enabling advances in artificial intelligence (AI) and high-performance computing (HPC). As these systems grow more sophisticated, ensuring chip security is essential to protect the data they process. This thesis evaluates open-source security features in RISC-V-based systems using adversarial testing, focusing on memory protection mechanisms such as Physical Memory Protection (PMP) and enhanced PMP (ePMP) within the context of Secure Boot. Using OpenTitan's Earlgrey System on Chip (SoC) [1] [2] [3] as a case study, selected protections are systematically weakened or removed to quantify each mechanism's contribution and derive practical recommendations for strengthening Hardware Root of Trust (HRoT) designs.

Keywords– High Performance Computing (HPC), RISC-V platforms, artificial intelligence (AI), System on Chip (SoC), Hardware Root of Trust (HRoT), Physical Memory Protection (PMP)

1 INTRODUCTION - CONTEXT OF THE THESIS

MODERN semiconductor designs increasingly embed security features directly in hardware so that sensitive operations can be protected with minimal performance penalties.

The open RISC-V instruction set architecture (ISA) [6]

-
- E-mail de contacte: pauromaguera1@gmail.com
 - Menció: Enginyeria de Computadors
 - Treball tutoritzat per: Marta Baldrís Calmet (Openchip and Software Technologies S.L.) Raimon Casanova Mohr (Departament de Microelectrònica i Sistemes Electrònics)
 - Curs 2025/26

accelerates this shift by enabling community-driven security extensions and cryptographic instructions that, together with dedicated accelerators and on-chip hardware roots of trust (HRoT), provide robust security with low overhead, safeguarding data and code integrity for end users.

In this thesis, we focus on the HRoT model in RISC-V systems, using OpenTitan as the primary case study [1]. Our analysis centers on the mechanisms that underpin a secure boot chain, particularly Physical Memory Protection (PMP) [8][9] and its enhanced variant ePMP [5], both of which enforce memory access controls from the earliest boot stages. By evaluating how these mechanisms contribute to overall system security, we aim to derive insights and recommendations for designing effective HRoT implementations in RISC-V SoCs.

HRoT functionality is typically integrated within a

System-on-Chip (SoC) and may include a dedicated core alongside hardened peripherals and memories to minimize the attack surface and resist software-based compromise. In this context, a clear separation of roles across boot stages, each with narrowly scoped privileges, enables more robust and reliable protection. In the remainder of this thesis we experimentally stress-test these mechanisms on OpenTitan’s Earlgrey SoC to assess their effectiveness against realistic adversaries and identify potential weaknesses or misconfigurations.

At the same time, advanced RISC-V designs use a general-purpose management core to orchestrate secure boot, power management, and low-level control for larger compute subsystems such as AI accelerators. These management cores often implement only the base PMP extension rather than ePMP. Understanding how security properties change when enhanced features such as machine-mode whitelisting (MMWP) are absent is therefore relevant beyond OpenTitan itself, in particular for heterogeneous SoCs where a relatively simple control core is expected to establish a strong hardware root of trust for more complex accelerators.

2 OBJECTIVES

The main objective of this thesis is to analyze and empirically prove the level of security of the open-source Earlgrey SoC from OpenTitan. In particular, the contribution of certain security mechanisms will be examined, such as enhanced Physical Memory Protection (ePMP) [5], in relation to the Secure Boot process, to verify how each function contributes to the overall security of the system.

To stress-test these mechanisms, we will use ablation tests. This involves intentionally modifying or removing specific security policies or functions in order to assess how vulnerable the SoC becomes to certain attack vectors. The main ablation test scenario to be included in this thesis is the removal of enhanced features of PMP, such as machine-mode whitelisting (MMWP) or rule locking bypass (RLB), to evaluate how the system behaves when only the base PMP is available.

This allows us to characterise which guarantees depend critically on ePMP-specific features and which can be approximated on platforms where a general-purpose control core, responsible for secure boot and power management of larger RISC-V subsystems, implements only base PMP. In doing so, we can identify which measures are most critical for protecting the system and establish a hierarchy of defenses based on their actual effectiveness.

3 PROJECT MANAGEMENT METHODOLOGY AND PLANNING

The project has been organized according to the Scrum methodology. The duration of tasks are planned in two-week iterative cycles, called sprints. Each task is expected to be completed within a sprint or, when necessary, decomposed into smaller sub-tasks to fit the two-week cycle. In addition, daily standup meetings are held with Openchip’s “Security Features” team to share progress and issues, while

Sprint Reviews are conducted to evaluate results, and retrospectives identify opportunities for improvement.

As mentioned earlier, each task has been structured to fit biweekly cadence, although some larger tasks have spanned two sprints, depending on issues encountered during execution. The timeline can be seen in the initial Gantt chart in Fig. 1, which shows the sequence and duration of the main activities.

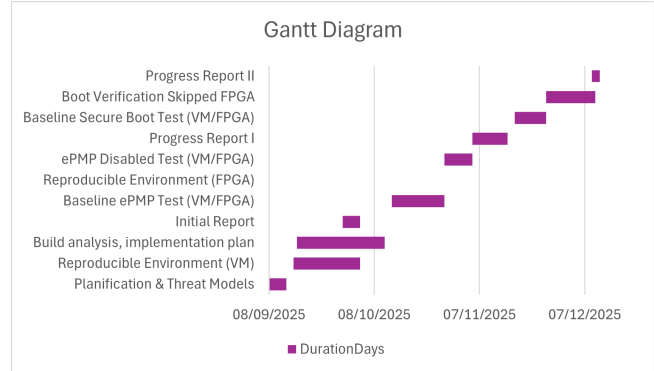


Fig. 1: Gantt diagram corresponding to the task execution of the project.

4 STATE OF THE ART

4.1 The OpenTitan Project

The experimental framework of this project relies on OpenTitan [1], the first open-source silicon Root of Trust (RoT) to reach the commercial market. It is a community-driven initiative supported by a several organizations, including lowRISC [11], ETH Zurich [12], Nuvoton [13], Google [14], Seagate [15], and Western Digital [16]. Within this ecosystem, lowRISC actively maintains the Ibex processor core, while Nuvoton has successfully taken the System-on-Chip (SoC) to its first tape-out (physical production), with the objective of integrating it as the HRoT in upcoming Google Chromebooks [17].

The development of such hardware is would not have been possible without the RISC-V architecture [6]. Its open and modular design has enabled the community to experiment with, extend, and validate new security functionalities, laying the necessary groundwork for ambitious projects like OpenTitan.

4.2 The Paradigm of Open-Source Security

The shift toward truly open-source hardware represents a significant milestone in semiconductor security. Unlike proprietary “black box” designs, open hardware allows every component of the Register Transfer Level (RTL) code design to be publicly audited, verified, and improved. This transparency is crucial for reducing the risk of hidden vulnerabilities and mitigating the threat of hardware Trojans [18], which are malicious modifications hidden within circuits, such as backdoors.

In a landscape where attacks are increasingly sophisticated and the technological supply chain remains vulnerable, open-source hardware introduces a new paradigm: trust is not blindly delegated to a single manufacturer but is built

collectively through open review and a diverse contributor base. This approach strengthens the foundational security of the system and fosters a more resilient ecosystem against potential threats.

4.3 The Earlgrey SoC

The primary target of our analysis is OpenTitan’s Earlgrey SoC. Apart from containing a low-power 32-bit RISC-V core (Ibex) equipped with distinct security features, the SoC integrates a suite of specialized security blocks. These include:

- **Cryptographic Engines:** Dedicated hardware for AES, HMAC, KMAC, and OTBN (OpenTitan Big Number accelerator for public-key cryptography).
- **Entropy Source:** A physical true random number generator (TRNG) for key generation.
- **Life-Cycle Controller:** A dedicated block governing provisioning states and debug access.
- **Key Manager:** Hardware for secure key derivation and storage.

Together with carefully partitioned non-volatile memory and peripheral isolation, these components implement a comprehensive Hardware Root of Trust. It is necessary to note that while these cryptographic and key management blocks are integral to the SoC’s overall security, this thesis restricts its experimental evaluation specifically to the early boot chain (Ibex, ePMP, and ROM firmware).

5 THREAT MODEL

We assume an adversary with physical but non-invasive access to the device, such as a device owner, service provider, or unauthorised third party. The attacker can either read or modify firmware images stored in non-volatile memory, or apply non-invasive fault mechanisms, for example, clock or voltage glitches during early boot.

The attacker’s primary goal is to bypass secure boot, for instance by executing unsigned or tampered firmware, or by redirecting control flow to unverified code. We also consider code-reuse attacks, such as Return-Oriented Programming (ROP), where an adversary exploits overly permissive memory configurations (eg., executable data sections) to construct malicious payloads from existing valid instructions. Invasive hardware attacks that require decapsulating the chip or probing internal wires are out of scope. This simplified model is aligned with the OpenTitan Lightweight Threat Model [19], but concentrates on non-invasive attacks against the secure-boot chain.

Gaining the ability to run arbitrary firmware is particularly attractive for an attacker because it effectively disables all higher-level protections implemented above the root of trust and enables persistent, stealthy compromise of the platform. Real-world incidents such as the Stuxnet[?] malware, which targeted industrial control systems by manipulating low-level code in programmable logic controllers, illustrate how control over early-stage or firmware-level execution can be used to cause long-term, hard-to-detect damage in critical infrastructures.

6 SYSTEM UNDER TEST: SECURE BOOT

Secure boot is the foundational security mechanism that ensures a device executes only trusted software. In complex RISC-V systems, the processor does not immediately load the operating system upon power-up. Instead, it follows a staged boot process known as a **Chain of Trust**.

In this model, the system begins by executing a small, immutable piece of code embedded directly in the silicon, known as ROM (Read-Only Memory) which constitutes the Root of Trust. This code authenticates the next stage of firmware using cryptographic signatures before granting it execution privileges. This process is repeated on each stage, verifying the next, until the full operating system is loaded. If any link in this chain is broken (eg., a signature check fails), the boot process halts, preventing the system from running compromised code.

6.1 Boot Stages in OpenTitan

OpenTitan implements this concept through three primary stages, dividing responsibilities between the manufacturer (*Silicon Creator*) and the user (*Silicon Owner*):

1. **ROM (Stage 0):** The immutable Hardware Root of Trust. Embedded directly in silicon, this code cannot be modified post-manufacture. Its primary responsibilities are to establish the initial security defenses (configuring ePMP), perform essential hardware initialization, and cryptographically verify the signature of the next stage (ROM_EXT) before transferring control.
2. **ROM_EXT (Stage 1):** The “ROM Extension” is stored in mutable eFlash and is managed by the Silicon Creator. It acts as a secure bridge between the immutable ROM and the owner-controlled firmware. This stage initializes complex hardware features not handled by the Mask ROM and authenticates the first stage of the owner’s firmware (BL0).
3. **BL0 (Stage 2):** The first boot stage signed by the Silicon Owner. This stage marks the transition of control from the manufacturer to the device owner. It performs platform-specific setup, such as configuring the pinmux and clocks, and is responsible for verifying and loading higher-level system software, such as the operating system kernel or a standard bootloader like OpenSBI.

Figure 2 illustrates the secure boot stack, highlighting the critical hardware isolation between the *Silicon Creator* (Machine Mode) and the *Silicon Owner*. As execution ascends, strict boundaries and separation prevent higher-level software from compromising the Root of Trust.

6.2 Memory Protection

OpenTitan’s 32-bit Ibex RISC-V core implements the enhanced Physical Memory Protection extension (ePMP). We first describe the base RISC-V Physical Memory Protection (PMP) in detail and then explain how ePMP strengthens early boot security and machine mode enforcement on this SoC.

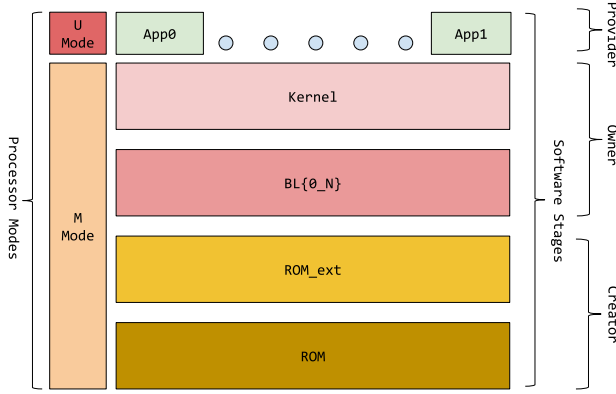


Fig. 2: Secure Boot Diagram

6.2.1 PMP

To support secure processing and contain software faults, it is essential to restrict the physical addresses that software can access. The Physical Memory Protection (PMP) unit provides per-hart (hardware thread) controls that allow Machine Mode to specify access permissions (Read, Write, Execute) for defined physical memory regions. By configuring these PMP entries, the system enforces fine-grained isolation, determining which areas of the physical address space are accessible to lower-privileged software, thereby improving overall memory safety and robustness.

While the RISC-V standard allows flexibility in implementation, OpenTitan's Ibex core specifically includes 16 PMP entries, enabling the definition of up to sixteen distinct protected regions. The protection standard relies on two types of Control and Status Registers (CSRs):

6.2.1.1 pmpaddrN registers store the physical address values that define the boundaries of PMP-protected regions. There is one `pmpaddr` register for each PMP entry implemented by the core. These registers encode the physical address shifted right by two bits, allowing for a minimum granularity of 4 bytes. Although the RISC-V specification allows for wider physical addresses to support complex memory management, OpenTitan's Ibex core utilizes a standard 32-bit physical address space, and these registers are implemented to match that width.

6.2.1.2 pmpcfgN Each `pmpcfg` CSR is four bytes wide and encodes four PMP entries (one byte per entry).

The configuration registers pack settings for four PMP entries into a single 32-bit CSR (one byte per entry). For the 16 entries in Ibex, registers `pmpcfg0` through `pmpcfg3` are used. Each byte defines the permissions (R/W/X), the lock bit (L), and the address-matching mode (A) for a specific region. Table 1 details the bit layout.

TABLE 1: ONE-BYTE LAYOUT OF A `PMP_CFG` ENTRY

Bits	Field	Allowed	Meaning / effect
7	L	0, 1	1: lock entry and enforce permissions on M-mode. 0: not locked (base PMP enforces on S/U only).
6–5	(reserved)	must be 0	Reserved. Writes should be zero (WARL).
4–3	A[1:0]	00, 01, 10, 11	Address-matching mode.
2	X	0, 1	Execute permission for the matched region.
1	W	0, 1	Write permission for the matched region.*
0	R	0, 1	Read permission for the matched region.

* The combination R=0 and W=1 is reserved in PMP implementations treat it as disallowing writes (i.e., equivalent to R=0, W=0).

6.2.1.3 PMP address-matching modes (A field) These are the address-matching modes defined by RISC-V PMP. Each mode trades off granularity versus the number of PMP entries consumed to describe a region.

- 1. OFF (00):** The entry is disabled. However, its address register can still serve as the lower bound for a subsequent TOR entry.
- 2. TOR (01):** *Top of Range.* Defines a range $[addr_{i-1}, addr_i)$. This mode allows for arbitrary region sizes but typically consumes two PMP entries (one for the start, one for the end).
- 3. NA4 (10):** *Naturally Aligned 4-byte.* Matches a single 4-byte word at the specified address. Useful for protecting individual registers or flags.
- 4. NAPOT (11):** *Naturally Aligned Power of Two.* Defines a region of size 2^k bytes (where $k \geq 3$). Efficient for fixed-size memory blocks like RAM or Flash partitions, consuming only a single entry.

6.2.1.4 The mcause CSR is a Machine-Mode Control and Status Register that records the specific reason for the most recent trap taken into M-mode. A trap can be either an interrupt (asynchronous, like a timer) or an exception (synchronous, like an illegal instruction). In the context of PMP, `mcause` is the primary diagnostic tool. When an access violates permissions, the hardware raises a specific exception code (e.g., `kIbexExcInstrAccessFault` for execution violations). The exception handler reads this code to determine the nature of the fault.

Crucially, the handler also relies on the `mepc` (**M**achine **E**xception **P**rogram **C**ounter) CSR, which hardware updates automatically to hold the address of the instruction that caused the trap. To resume execution after handling a fault, the software must manually advance `mepc` to the next instruction, otherwise, the processor will return to the same faulting instruction, creating an infinite trap loop.

6.2.1.5 Representative exception causes (Ibex) In the context of PMP and secure boot evaluation, the following

Ibex exception codes are the most relevant. The names and numeric values correspond to the `ibex_exc_t` enumeration used by the firmware.

1. **`kIbexExcInstrAccessFault` (1)** Raised when an instruction fetch targets an address without execute permission, for example, PMP denies `X` or otherwise cannot be accessed.
2. **`kIbexExcIllegalInstrFault` (2)** Raised when the fetched instruction encoding does not correspond to any valid opcode for the implemented ISA. In this work, it is common to execute from regions prefilled with all ones (e.g., `0xFFFF...`). If execute permission is denied by PMP, the trap is an *instruction access fault*, if execution is permitted, the all-ones word decodes to an invalid instruction, raising this fault instead.
3. **`kIbexExcLoadAccessFault` (5)** Raised when a load violates access policy, for example, PMP denies `R` or targets an inaccessible region.
4. **`kIbexExcStoreAccessFault` (7)** Raised when a store/AMO violates access policy, for example, PMP denies `W` or targets an inaccessible region.

Therefore, the `mcause` register provides a clear means of identifying which exception was raised by a given instruction fetch or data access during our experiments with PMP-configured regions.

6.2.2 ePMP

Standard PMP suffers from a “default-allow” posture in Machine Mode: unless an entry is explicitly locked, M-mode ignores the protection rules. Furthermore, unmatched addresses are often accessible, creating dangerous gaps during the complex initialization of early boot.

The Enhanced PMP (ePMP) extension (standardized as `Smepmp` [5]) addresses these vulnerabilities by introducing the `mseccfg` CSR. This register allows the system to enforce a strict “whitelist-by-default” policy and provides mechanisms to safely update locked rules, ensuring robust protection from the very first instruction.

6.2.2.1 The `mseccfg` CSR The `mseccfg` (Machine Security Configuration) register is the control center for these new policies. Accessible only in M-mode, it defines bits that alter PMP’s default behavior while maintaining backward compatibility with base PMP when cleared. On Ibex/OpenTitan, only a specific subset of these bits is used:

6.2.2.2 Rule Locking Bypass (`mseccfg.RLB`) This bit provides a controlled, temporary override of the locked state so that boot code can safely evolve PMP after first constraining M-mode:

- **If `RLB=1`:** Software may modify or remove entries even when `pmpcfg.L=1`, allowing the ROM to lock down early, verify the next stage, and then reconfigure cleanly.

- **If `RLB=0` and any entry has `L=1`:** `RLB` becomes “sticky” at 0 and further writes to set it are ignored until a PMP reset. This prevents late-stage code from reopening a bypass once protections are committed. This stickiness is a defense-in-depth choice to avoid compromising earlier security features.

6.2.2.3 Machine-Mode Whitelist Policy (`mseccfg.MMWP`) This bit changes the default outcome for M-mode when no PMP entry matches the target address:

- **If `MMWP=1`:** unmatched accesses are denied (true whitelist by default).
- **If `MMWP=0`:** Base PMP semantics apply. Consequently, unmatched accesses are allowed in Machine Mode (default-allow).

In practice, platforms enable `MMWP` early (or wire it high via RTL) to avoid reintroducing default-allow windows during the boot chain.

6.2.2.4 Machine-Mode Lockdown (`mseccfg.MML`) This bit reinterprets the PMP lock mechanism to separate enforcement between Machine Mode and lower privilege levels (User or Supervisor modes). As OpenTitan’s standard boot flow does not utilize this feature, it is excluded from our experimental scope.

6.2.2.5 Relevance to Secure Boot During the ROM and ROM.EXT stages, strict adherence to the Principle of Least Privilege is required: only the minimal verified code region should be executable. `MMWP` enforces this by eliminating default-allow gaps, while `RLB` provides a mechanism to lock protections early yet safely evolve the memory layout once the next stage is authenticated. This combination yields a significantly more robust secure-boot posture than base PMP alone, while maintaining backward compatibility for our ablation experiments.

6.2.3 OpenTitan’s ePMP configuration

This section analyzes the transition of OpenTitan’s ePMP configuration from a minimal reset state to a fully-fledged runtime policy.

6.2.3.1 Hardwired Configuration At reset, Ibex loads a minimal ePMP state from RTL constants. The arrays `PmpCfgRst[16]` and `PmpAddrRst[16]` predefine a small set of regions (ROM, MMIO, Debug ROM) while leaving the remainder *OFF*. The machine security configuration CSR resets to `mseccfg.RLB=1` and `mseccfg.MMWP=1`. Thus, unmatched addresses are denied by default in M-mode, and the ROM is allowed to refine or remove locked entries during early boot. Leaving unused entries *OFF* is therefore safe: with `MMWP=1` there are no default-allow gaps at reset.

6.2.3.2 Hardwired Configuration At reset, Ibex loads a minimal configuration from RTL constants, covering only essential regions (ROM, MMIO, Debug) while leaving the rest *OFF*. Crucially, `mseccfg` resets to `RLB=1`

TABLE 2: ROM-STAGE EPMP ENTRIES

Entry	Description	Permissions	Addressing Mode
1	ROM TEXT	RX	TOR
2	ROM	R	NAPOT
5	eFlash	R	NAPOT
11	MMIO	RW	TOR
14	Stack Guard	<none>	NA4
15	RAM	RW	NAPOT

Note: ROM_EXT remains non-executable in this state until explicitly unlocked after successful signature verification.

and `MMWP=1`. This ensures that unmatched addresses are denied by default—closing potential gaps, while allowing the ROM to safely refine locked entries during the boot sequence.

6.2.3.3 Firmware Initialization Immediately following reset, the processor begins execution at the assembly entry point `rom_start.S`. This startup code performs minimal initialization and ends up calling the function `rom_epmp_init`. It is important to note that prior to this function’s execution, critical resources such as eFlash and SRAM are not yet mapped by any PMP entry, though they remain inaccessible due to `MMWP=1`.

6.2.3.4 Firmware Initialization Immediately following reset, the processor begins execution at the assembly entry point `rom_start.S`. This startup code performs minimal initialization before invoking the assembly function `rom_epmp_init`. Prior to this function’s execution, critical resources such as eFlash and SRAM are not yet mapped by any PMP entry although they remain inaccessible due to the default-deny policy enforced by `MMWP=1`. The initialization routine then establishes the runtime memory layout summarized in Table 2. Key regions include **ROM TEXT** (RX), **ROM** data (R), and **MMIO** (RW). Notably, the **eFlash** is mapped as Read-Only to allow signature verification, but remains non-executable to prevent premature code execution.

6.2.3.5 Enabling ROM_EXT Execution Upon successful signature validation, the ROM updates the ePMP configuration to grant execution permissions to the `ROM_EXT` text section. This update, performed by `rom_epmp_unlock_rom_ext_rx`, relies on `mseccfg.RLB=1` to modify previously locked entries. Once control is transferred, subsequent stages can clear this bit to prevent further changes. This staged approach ensures that the flash memory remains non-executable until cryptographic verification is complete, maintaining a strict security posture.

7 EXPERIMENTAL METHODOLOGY

The experimental testing is conducted using Verilator, a cycle-accurate simulator that compiles the digital RTL design (SystemVerilog code) of OpenTitan’s Earlgrey SoC

into a high-performance C++ model. This approach enables efficient emulation of the hardware platform, offering a balance between execution speed and functional fidelity suitable for firmware-level security analysis.

7.1 Test environment

The OpenTitan development ecosystem provides a toolchain organized by **Bazel** to build both the hardware simulation and the firmware images. **FuseSoC** manages the RTL dependencies, which are translated by **Verilator** into a cycle-accurate C++ executable (`Vchip_sim_tb`). Finally, **Opentitantool** interfaces with this binary to load firmware images (ROM, Flash, OTP) and capture UART output during execution.

It is important to note that while Verilator is faster than traditional logic simulators, the cycle-accurate nature of the emulation still imposes significant computational overhead. Consequently, complex test sequences - particularly those involving cryptographic operations or extended boot flows - resulted in high simulation times, with some individual tests requiring upwards of fifteen minutes to complete in an 8-core Intel Xeon Gold 6130 CPU at 2.10GHz.

Regarding the experimental methodology, we utilize two distinct instances of this simulation environment. The first instance serves as the baseline, running the unmodified SoC design to establish expected behavior. The second instance is used for experimental ablation, where specific security policies are removed, relaxed, or altered to quantify their impact on the system’s overall security posture.

An emulation approach using Field-Programmable Gate Arrays (FPGAs) was initially considered, however, given schedule constraints and late hardware availability, we considered the approach beyond the scope of this thesis.

7.2 Experimental Design And Implementation

To systematically verify memory protections, we cannot simply rely on standard software assertions, as a security violation triggers a hardware trap (exception) rather than a return code. OpenTitan addresses this with a baseline test, `rom_epmp_test.c`, which probes specific memory locations and compares the resulting exception with an expected value. However, this test is limited in scope, as it is constrained primarily to execute-only accesses and does not cover the full range of memory kinds and access types required for a complete security verification.

To overcome these limitations, we developed a custom test suite that operates as a direct replacement for the standard Mask ROM firmware. By linking against the ROM linker script and utilizing the standard reset vector, our test executes in Machine Mode (M-Mode) immediately following system reset. This privileged context is essential for verifying the initial ePMP configuration and the locking mechanisms. Instead of proceeding with the standard boot flow -which would attempt to verify and jump to the `ROM_EXT` in Flash- this test changes the control flow to execute our verification routines directly from the ROM text region.

Furthermore, for these experiments, the eFlash memory is left in its erased state (filled with `0xFFFFFFFF`). This provides a predictable target for testing invalid instruction

execution, as such value is not a valid RISC-V instruction. Crucially, this also acts as a **fail-safe**: if a security gap were to erroneously permit execution in this region, the processor would immediately trap on the first instruction fetch rather than continuing to execute potentially undefined memory regions, ensuring that any failure is contained and observable. Finally, from a simulation perspective, this strategy reduces computational overhead by treating the flash as a constant value that the simulator can serve efficiently, therefore avoiding latency associated with loading large binary images or modeling complex memory content.

7.2.1 Custom Test Suite

To support this approach we adapted the existing test infrastructure to recover from intentional faults. While the standard OpenTitan exception handler is capable of recovering from execution faults it treats load or store exceptions as fatal errors. Consequently we expanded the handler logic to intercept these additional exception types and resume execution.

7.2.1.1 Fault Injection and Recovery We introduced two helper primitives, `read32` and `write32`, which attempt 32-bit accesses to a target address. Before executing the operation, these functions reset a global state variable, `exception_received`, to a sentinel value (`kIbexExcMax`). If the access triggers a hardware trap due to ePMP enforcement, control is automatically transferred to the exception handler.

The verification logic is encapsulated in a generic `TestAccess` routine (Algorithm 1). This routine attempts a specific memory operation (Read, Write, or Execute) using the helpers and verifies that the hardware response matches the expected security policy.

Algorithm 1 Memory Protection Verification Routine

```

1: Global volatile exception_received
2: procedure VERIFYREGION(base, size, op, expect)
3:   start  $\leftarrow$  base
4:   end  $\leftarrow$  base + size - 4
5:   for addr  $\in$  {start, end} do
6:     ACCESS(addr, op, expect)
7:   end for
8: end procedure
9: procedure ACCESS(addr, op, expect)
10:  exception_received  $\leftarrow$  kIbexExcMax
11:  try perform op (read / write / execute) on addr
12:  if Hardware Trap Occurs then
13:    ROM_EXCEPTION_HANDLER
14:  end if
15:  if exception_received  $\neq$  expect then
16:    Report Failure
17:  end if
18: end procedure
19: procedure ROM_EXCEPTION_HANDLER
20:  mcause  $\leftarrow$  CSR_READ(mcause)
21:  exception_received  $\leftarrow$  mcause
22:  CSR_WRITE(mepc, next_instruction)
23: end procedure

```

7.2.1.2 Exception Handling The existing exception handler was extended to support load and store exceptions in addition to execution faults. Upon entry, the handler reads the `mcause` CSR to identify the exception type and records it in the global `exception_received` variable.

To allow the test to proceed, the handler must advance the Machine Exception Program Counter (`mepc`) to the next instruction. Crucially, since the Ibex core supports the RISC-V Compressed Extension (C), instructions may be either 16-bit or 32-bit wide. Simply incrementing the PC by a fixed 4 bytes resulted in misalignment or infinite trap loops. Debugging this was challenging since the failures were intermittent and depended on the specific instruction encoding. Therefore, the handler checks the lower bits of the faulting instruction to determine its length and increments `mepc` by either 2 or 4 bytes accordingly.

7.2.1.3 Region-Specific Verification Dedicated verification functions were implemented for each distinct memory region (e.g., ROM, eFlash, RAM, MMIO). Given that an exhaustive scan of the entire address space would be prohibitively slow in a cycle-accurate simulator like Verilator, our verification strategy focuses on critical addresses—specifically the start and end words of each defined region—where configuration errors are most likely to manifest.

Within these functions, the test logic probes these boundaries using the appropriate access helpers. The outcome is then validated against the hardcoded expected exception (e.g., `kIbexExcInstrAccessFault` for non-executable regions), and any discrepancy triggers a failure report. Additionally, specific functions were implemented to verify the “sticky” semantics of the `mseccfg` register, ensuring that critical security policies cannot be reverted by software once established.

7.2.1.4 Watchdog Management Finally, a mechanism to disable the watchdog timer was introduced to prevent unintended system resets during prolonged test execution. Unlike the original short-duration tests, our extended testing exceeded the default watchdog window. To debug the root cause of unexpected resets, the reset cause register was analyzed, indicating a reset triggered by an interrupt. Consequently, the watchdog is now explicitly disabled at the beginning of the test sequence to ensure stability.

7.2.2 Verification Strategy

Based on the described methodology, we defined a set of experiments to characterize the ePMP implementation. These tests are driven from the ROM/ROM_EXT context and validate four distinct security properties.

7.2.2.1 Access control for configured regions First, we validate the permissions for regions explicitly covered by ePMP entries (Table 2). The test issues *execute*, *load*, and *store* probes to both interior addresses and boundary words of each region.

To pass this verification, the hardware must strictly enforce the defined permissions. For example, the **FLASH** region is configured as Read-Only, therefore, the test expects load operations to succeed, while store and execute attempts must trigger `kIbexExcStoreAccessFault`

8.2.1 Failure of Default-Deny Policy

With MMWP hardwired to 0, our probes to unconfigured memory regions (specifically the unmapped eFlash) completed without raising exceptions. This confirms that without ePMP, the system defaults to a "blacklist" approach, where any memory not explicitly covered by a PMP entry remains accessible to Machine Mode.

As illustrated in Figure 4, under this configuration, accesses to the unconfigured area bypass the protection logic and complete successfully, standing in clear contrast to the default-deny behavior observed when MMWP is enabled.

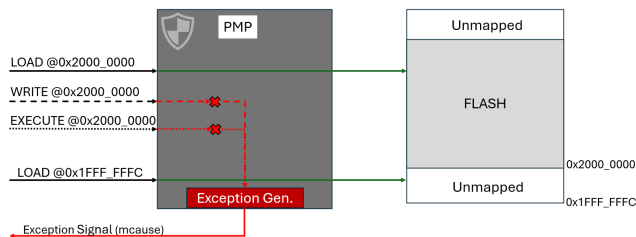


Fig. 4: High-level abstraction of the logical flow of the Region-Specific Verification, now showcasing PMP behaviour with $\text{MMWP} = 0$.

8.2.2 Inability to Harden ROM Permissions

During the execution of the ablation test suite, a critical anomaly was detected in the ROM region verification. While most security checks aligned with expectations for a PMP-only system, the test case asserting that the ROM read-only data section (`.rodata`) must be non-executable failed consistently. Contrary to the secure boot policy, the hardware permitted code execution from this data-only region.

Initially suspected to be a software bug in the test suite, further investigation revealed that this behavior was a direct architectural consequence of disabling ePMP. At reset, the hardware loads a default PMP entry covering the entire ROM address space with Locked, Read, and Execute (L/R/X) permissions.

This permissive default is architecturally necessary for two reasons. First, the processor must be able to fetch the reset vector immediately upon power-on, otherwise, an instruction access fault would occur before the first instruction is executed. Second, while the hardware knows the physical size of the ROM, it is agnostic to the software layout. The boundary between executable code (`.text`) and constant data (`.data`, `.rodata` among others), is determined at compile time. Consequently, the RTL cannot enforce a granular restriction without risking compatibility with future firmware builds, forcing it to map the entire ROM size as executable.

In the standard boot flow, the `rom_epmp_init` function resolves this by reading the firmware’s linker symbols to identify the exact code boundary. It then attempts to remove the execute permission from the original wide entry and add a new, granular rule granting execution only to the `.text` section.

However, with RLB forced to 0 in our ablation study, the initial "wide" PMP entry loaded at reset is immediately and

permanently locked. Consequently, the firmware’s attempt to restrict the data section to Read-Only is blocked by hardware. Our tests confirmed that the ROM data section remained executable throughout the test, exposing a potential attack surface for Return-Oriented Programming (ROP) attacks. This finding highlights that RLB is a necessity for transitioning from a permissive boot state to a secure runtime configuration.

8.3 Identified Attack Vector: The Initialization Gap

The results from the ablation study reveal a critical attack vector that emerges during the early boot phase. Specifically, a significant window of vulnerability exists between the hardware reset and the completion of the `rom_epmp_init` function, which is responsible for configuring the full PMP layout.

During this interval, the system relies on the minimal PMP configuration hardwired at reset, which leaves large portions of the memory map - including the eFlash - unconfigured. As demonstrated in our previous testing, when $MMWP=0$ base PMP semantics apply, therefore, Machine Mode is permitted to execute from any address not explicitly covered by a PMP entry. This creates a substantial risk: an attacker could induce a control-flow glitch (eg., via voltage fault injection, as outlined in our Threat Model) to redirect the Program Counter (PC) to the unmapped eFlash region before initialization completes, the processor will successfully execute the code found there, even if it is malicious. This effectively bypasses the entire Secure Boot chain, allowing the execution of arbitrary, unverified payloads before the Root of Trust has established its defenses.

9 CONCLUSION

This thesis analyzed and evaluated the security mechanisms of open-source RISC-V Hardware Roots of Trust, using OpenTitan’s EarlGrey SoC as a case study. Through a rigorous experimental methodology utilizing cycle-accurate simulation and adversarial testing, we quantified the specific security contributions of the Enhanced Physical Memory Protection (ePMP) extension.

The results conclusively demonstrate that the standard RISC-V PMP extension, while sufficient for basic isolation in mature operating systems, lacks the necessary primitives to secure the earliest stages of the boot chain, specially if modularity and hardware reusability is a requirement. Our experiments revealed that disabling ePMP features exposes the system to two critical attack vectors:

1. **The Initialization Gap:** The absence of the Machine-Mode Whitelist Policy (MMWP) creates a window of vulnerability between reset and firmware initialization. During this window, unmapped memory regions are executable by default, allowing potential control-flow glitches to bypass the root of trust entirely and allow malicious firmware to execute.
2. **Persistent Over-Privilege:** The absence of the Rule Locking Bypass (RLB) prevents the firmware from adhering to the Principle of Least Privilege. Because the

hardware must be permissive at reset to allow booting, the inability to later lock down these permissions results in read-only data sections remaining executable throughout the device's runtime.

These findings highlight a fundamental tension between system initialization and runtime security: a system must be open to boot, but closed to be secure. ePMP resolves this by allowing a controlled transition from a "boot-friendly" state to a "security-hardened" state. Conclusively, this work suggests that general-purpose cores implementing only base PMP aren't suited for acting as a Hardware Root of Trust unless supplemented by external hardware logic. For heterogeneous SoCs relying on a management core, the adoption of ePMP should be considered a mandatory requirement.

Future work could extend these findings by validating the identified attack vectors on physical FPGA implementations or by exploring the interaction between ePMP and other RISC-V security elements, such as Physical Memory Attribution (PMA) or Control Flow Integrity (CFI). However, the evidence presented here serves as a strong recommendation for system architects, to build a resilient Hardware Root of Trust in RISC-V, memory protection must be whitelist-based and dynamic from the very first clock cycle.

ACKNOWLEDGEMENTS

[TBD]

REFERENCES

- [1] *OpenTitan: Open Source Silicon Root of Trust*. Disponible: <https://opentitan.org>
- [2] A. Meza, C. Tunc, P. Nasahl, and J. Grossklags, "Security Verification of the OpenTitan Hardware Root of Trust," doi: 10.1109/MSEC.2023.3255363.
- [3] A. Musa, E. Parisi, L. Barbierato, A. Acquaviva, and F. Barchi, "End-to-end Integration of OpenTitan Security Features in a Pure RISC-V SoC," doi: 10.1109/SMACD61181.2024.10745397.
- [4] S. K. Saha, A. N. Butka, M. K. Ahmed, and C. Bobda, "OpenTitan based Multi-Level Security in FPGA System-on-Chips," doi: 10.1109/ICFPT59805.2023.00056.
- [5] *Secure Memory Protection Extension (PDF)*, Github: [riscvarchive/riscv-tee](https://github.com/riscvarchive/riscv-tee) Project. Disponible: <https://github.com/riscvarchive/riscv-tee/blob/191b563b08b31cc2974d604a3b670d8666a2e093/Smepmp/Smepmp.pdf> (Accedit: 2025-09-23)
- [6] *The RISC-V Instruction Set Manual (Unprivileged ISA)*. RISC-V International. Disponible: <https://riscv.org/technical/specifications/> (Accedit: 2025-11-15)
- [7] *Hardware Root of Trust*. Rambus Blog. Disponible: <https://www.rambus.com/blogs/>
- [8] *RISC-V Memory Protection: Diving deep into the complexities*. Medium (Karthik B K). Disponible: <https://medium.com/@talktokarthikbk/risc-v-memory-protection-diving-deep-into-the> (Accedit: 2025-11-15)
- [9] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture (inclou PMP)*. RISC-V International. Disponible: <https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf> (Accedit: 2025-11-15)
- [10] *Verified Boot Crypto*. Chromium OS Design Docs. Disponible: <https://www.chromium.org/chromium-os/chromiumos-design-docs/verified-boot-crypto/> (Accedit: 2025-11-15)
- [11] *OpenTitan Partnership Makes History: First Open-Source Silicon to Reach Commercial Availability*. lowRISC News (13 Feb 2024). Disponible: <https://lowrisc.org/news/opentitan-commercial-availability/> (Accedit: 2025-10-04)
- [12] "A booting computer is as vulnerable as a newborn baby". ETH Zürich News (5 Nov 2019). Disponible: <https://ethz.ch/en/news-and-events/eth-news/news/2019/11/project-opentitan.html> (Accedit: 2025-11-15)
- [13] *Nuvoton Develops OpenTitan®-based Security Chip as Commercial Product*. Press Release (30 May 2024). Disponible: <https://www.nuvoton.com/news/news/all/TSNuvotonNews-000514/> (Accedit: 2025-11-15)
- [14] *Fabrication begins for production OpenTitan silicon*. Google Open Source Blog (6 Feb 2025). Disponible: <https://opensource.googleblog.com/2025/02/fabrication-begins-for-production-opentitan-s.html> (Accedit: 2025-11-15)
- [15] *Seagate Designs RISC-V Cores to Power Data Mobility and Trustworthiness*. Seagate Article. Disponible: <https://www.seagate.com/stories/articles/seagate-designs-risc-v-cores-to-power-data-mo> (Accedit: 2025-11-15)
- [16] *hardware-root-of-trust/* (Accedit: 2025-11-15)
- [17] *Keystone: An Open Framework for Architecting Trusted Execution Environments*. EuroSys 2020. Disponible: https://www.shwetaashinde.org/publications/keystone_eurosys20.pdf (Accedit: 2025-11-15)

- [16] *Western Digital Collaborates with lowRISC and Google to Increase Security Transparency in Data-Centric Platforms*. Western Digital Press Release (5 Nov 2019). Disponible: <https://www.westerndigital.com/company/newsroom/press-releases/2019/2019-11-05-western-digital-collaborates-with-lowrisc-and-google-to-increase-security-t> (Accedit: 2025-11-15)
- [17] *OpenTitan production silicon and early integrations*. lowRISC News (20 Feb 2025). Disponible: <https://lowrisc.org/news/the-worlds-first-open-source-security-chip-hits-production-with-google/> (Accedit: 2025-11-15)
- [18] *Hardware Trojans in Chips: A Survey for Detection and Prevention*. Sensors (MDPI), vol. 20, no. 18, 5165 (2020). Disponible: <https://www.mdpi.com/1424-8220/20/18/5165> (Accedit: 2025-11-15)
- [19] *OpenTitan Lightweight Threat Model*. Project documentation. Disponible: https://opentitan.org/book/doc/security/threat_model/index.html (Accedit: 2025-11-15)
- [20] *Design Approaches and Architectures of RISC-V SoCs (referència a Sv32)*. RISC-V International Blog. Disponible: <https://riscv.org/blog/design-approaches-and-architectures-of-risc-v-socs/> (Accedit: 2025-11-15)

APPENDIX